

GUIA COMPLETO

Skills & Plugins do Claude Code

Material de Estudo e Referência Rápida

12 Repositórios · 35 Skills · 55+ Comandos · 90+ Agents · 3 MCP Servers

Componente	Quantidade	Local
Skills de Projeto	35	.claude/skills/
Plugins Marketplace	5	~/.claude/plugins/marketplaces/
Comandos Globais	55+	~/.claude/commands/
Agents Globais	12+	~/.claude/agents/
MCP Servers	3	~/.mcp.json
CLIs Instalados	3	ctx7, notebooklm, bun

Gerado em: 14/03/2026 20:19

Sumário

1. Visão Geral e Arquitetura
2. claude-mem — Sistema de Memória Persistente
3. n8n-mcp — Automação de Workflows
4. obsidian-skills — Formatos do Obsidian
5. get-shit-done — Desenvolvimento Guiado por Specs
6. superpowers — Biblioteca de Padrões de Desenvolvimento
7. awesome-claude-code — Comandos da Comunidade
8. ui-ux-pro-max — Inteligência de Design UI/UX
9. notebooklm-py — Google NotebookLM
10. Anthropic Skills — Coleção Oficial
11. Context7 — Documentação em Tempo Real
12. Ralph — PRDs e Execução Autônoma
13. ruflo/claude-flow — Orquestração Enterprise
14. Base de Dados — Referência Rápida de Todos os Comandos
15. Compatibilidade por Plataforma

1. Visão Geral e Arquitetura

O que são Skills, Plugins e MCP Servers?

O Claude Code possui um ecossistema extensível com três tipos principais de extensões:

Skills (.claude/skills/)

Arquivos SKILL.md que ensinam ao Claude novas capacidades. São ativadas automaticamente quando o contexto da conversa corresponde à descrição da skill, ou manualmente via /nome-da-skill. Cada skill contém instruções, referências e opcionalmente scripts auxiliares.

Plugins Marketplace (~/.claude/plugins/marketplaces/)

Pacotes completos que incluem skills, comandos, agents, hooks e configurações MCP. São instalados globalmente e ficam disponíveis em todos os projetos. Podem incluir hooks de lifecycle que injetam contexto automaticamente.

MCP Servers (~/.mcp.json)

Servidores que implementam o Model Context Protocol, fornecendo ferramentas (tools) que o Claude pode chamar. Rodam como processos separados e são configurados em ~/.mcp.json. Permitem integração com serviços externos.

Comandos Slash (~/.claude/commands/)

Arquivos .md que definem prompts reutilizáveis. Invocados com /categoria:nome. Podem estar no nível global (~/.claude/commands/) ou do projeto (.claude/commands/).

Agents Customizados (~/.claude/agents/)

Definições de subagentes especializados em .md. Podem ser invocados pelo Claude para delegar tarefas específicas a agentes com instruções e modelos customizados.

Onde cada coisa fica instalada:

Tipo	Local	Escopo
Skills de Projeto	.claude/skills/	Apenas este projeto
Skills Globais	~/.claude/skills/	Todos os projetos
Plugins	~/.claude/plugins/marketplaces/	Global
Comandos Globais	~/.claude/commands/	Global
Comandos de Projeto	.claude/commands/	Apenas este projeto

Tipo	Local	Escopo
Agents Globais	~/.claude/agents/	Global
MCP Servers	~/.mcp.json	Global
Hooks	~/.claude/settings.json ou plugin hooks.json	Global

2. claude-mem — Sistema de Memória Persistente

Repositório: thedotmack/claude-mem | **Versão:** 10.5.5

Local: ~/.claude/plugins/marketplaces/thedotmack/

O que faz

O claude-mem é um sistema de memória persistente que captura automaticamente tudo que o Claude faz durante sessões de codificação, comprime com IA, e injeta contexto relevante em sessões futuras. Ele resolve o problema fundamental de que cada sessão do Claude Code começa "do zero" — com o claude-mem, o Claude lembra do que foi feito antes.

Quando usar

- Quando precisa que o Claude lembre de decisões anteriores
- Quando quer buscar como algo foi resolvido em sessões passadas
- Para projetos de longo prazo com múltiplas sessões
- O sistema funciona automaticamente via hooks — não precisa ativar manualmente

Skills e Comandos

Skill/Comando	Trigger	Descrição	Quando Usar
/mem-search	Automático ou manual	Busca nas memórias de sessões passadas usando workflow de 3 camadas: search → timeline → get_observations	"Como resolvemos X da última vez?", "O que fizemos na semana passada?"
/make-plan	Manual	Cria planos de implementação detalhados em fases usando subagentes para pesquisa de documentação e validação	Antes de iniciar uma feature complexa que precisa de planejamento
/do	Manual	Executa planos criados pelo make-plan usando subagentes para cada fase, com verificação e anti-patterns	Após ter um plano pronto, para executar fase por fase
/smart-explore	Manual	Exploração de código via AST (tree-sitter) — 4-8x mais eficiente que ler arquivos inteiros	Quando precisa entender código sem gastar muitos tokens

Hooks (automáticos)

Hook	Quando Dispara	O que Faz
SessionStart	Ao abrir sessão	Instala dependências, inicia worker, injeta contexto de sessões anteriores
UserPromptSubmit	A cada prompt do usuário	Inicializa sessão no sistema de memória
PostToolUse	Após cada uso de ferramenta	Captura observações sobre o que o Claude fez
Stop	Quando Claude para de responder	Gera resumo comprimido da atividade
SessionEnd	Ao fechar sessão	Marca sessão como completa no banco de dados

Arquitetura Técnica

- **Worker Service:** API HTTP na porta 37777 com interface web
- **SQLite:** Armazena sessões, observações, resumos
- **Chroma Vector DB:** Busca semântica + keyword search híbrida
- **MCP Tools:** search, timeline, get_observations
- **Tree-sitter:** Parsers para C, C++, Go, Java, JS, Python, Ruby, Rust, TypeScript

Exemplo de Uso

```
# Buscar como autenticação foi implementada antes
/mem-search
search(query="authentication", limit=20, project="meu-projeto")
# → Retorna tabela com IDs, timestamps, títulos

# Ver contexto ao redor de um resultado
timeline(anchor=11131, depth_before=5, depth_after=5)

# Buscar detalhes completos
get_observations(ids=[11131, 10942])
```

3. n8n-mcp — Automação de Workflows

Repositório: czlonkowski/n8n-mcp | **Versão:** 2.37.1

Local: MCP Server via npx n8n-mcp (configurado em ~/.mcp.json)

O que faz

Servidor MCP que conecta o Claude ao n8n, plataforma de automação de workflows. Fornece acesso a documentação de 1.239+ nodes de automação, permite buscar, validar e gerenciar workflows do n8n diretamente via Claude.

Quando usar

- Quando precisa criar ou editar workflows no n8n
- Para buscar documentação de nodes específicos do n8n
- Para validar configurações de workflow antes de deploy
- Para acessar templates de workflow prontos

Ferramentas MCP Disponíveis

Categoria	Ferramentas	Descrição
Discovery	search_nodes, get_node_info	Buscar e explorar 809 core + 430 community nodes
Configuration	get_node_essentials, get_examples	Detalhes de configuração, exemplos de uso
Validation	validate_config, validate_workflow	Validar configurações antes do deploy
Management	create_workflow, update_workflow	Criar e atualizar workflows (requer API key)
Templates	search_templates	Buscar entre 2.709+ templates de workflow

IMPORTANTE: Nunca edite workflows de produção diretamente. Sempre teste em ambiente de desenvolvimento primeiro.

Configuração

```
# Variáveis de ambiente necessárias para management:
export N8N_API_URL="https://seu-n8n.com/api/v1"
export N8N_API_KEY="sua-api-key"

# O servidor MCP já está configurado em ~/.mcp.json
```

Reinicie o Claude Code para ativar

4. obsidian-skills — Formatos do Obsidian

Repositório: kepano/obsidian-skills | Local: .claude/skills/

O que faz

Coleção de skills que ensinam o Claude a trabalhar nativamente com os formatos de arquivo do Obsidian — o popular app de notas. Cobre Markdown especial do Obsidian, Bases, JSON Canvas e CLI.

Skill	Ativação	Descrição	Quando Usar
obsidian-markdown	Automática ao editar .md do Obsidian	Cria/edita Obsidian Flavored Markdown com wikilinks [[link]], callouts, properties YAML, embeds	Criar notas, documentação, wikis no formato Obsidian
obsidian-bases	Ao trabalhar com .base files	Trabalha com Obsidian Bases — views, filters, formulas sobre dados estruturados	Criar databases/views no Obsidian
json-canvas	Ao trabalhar com .canvas files	Gerencia JSON Canvas — nodes (text, file, link, group) e edges (connections)	Criar mapas mentais, diagramas de fluxo no Obsidian
obsidian-cli	Manual	Operações via CLI no vault — listar, buscar, modificar notas	Automação em batch no vault Obsidian
defuddle	Ao extrair conteúdo web	Extrai markdown limpo de páginas web, removendo navegação, ads, etc. Reduz uso de tokens	Salvar conteúdo web como notas limpas

5. get-shit-done — Desenvolvimento Guiado por Specs

Repositório: gsd-build/get-shit-done | **Versão:** 1.22.4

Local: ~/.claude/ (global — commands, agents, hooks)

O que faz

Sistema completo de meta-prompting e context engineering para desenvolvimento guiado por specs. Organiza o trabalho em projetos → milestones → fases, com pesquisa, planejamento, execução e verificação estruturados.

Fluxo de Trabalho Típico

- 1. /gsd:new-project → Criar projeto com especificação
- 2. /gsd:new-milestone → Definir milestone do projeto
- 3. /gsd:plan-phase → Planejar fase de implementação
- 4. /gsd:research-phase → Pesquisar antes de executar
- 5. /gsd:execute-phase → Executar a fase planejada
- 6. /gsd:verify-work → Verificar que tudo funciona
- 7. /gsd:complete-milestone → Marcar milestone como completo

Comandos Principais (/gsd:*)

Comando	Categoria	Descrição
/gsd:new-project	Projeto	Criar novo projeto com especificação inicial
/gsd:new-milestone	Projeto	Criar milestone dentro do projeto
/gsd:complete-milestone	Projeto	Marcar milestone como concluído
/gsd:plan-phase	Fases	Planejar uma fase de implementação
/gsd:execute-phase	Fases	Executar uma fase planejada
/gsd:research-phase	Fases	Pesquisar antes de implementar
/gsd:validate-phase	Fases	Validar resultados de uma fase
/gsd:discuss-phase	Fases	Discutir uma fase com o Claude
/gsd:add-phase	Fases	Adicionar nova fase ao milestone
/gsd:debug	Execução	Debugging estruturado com metodologia
/gsd:quick	Execução	Task rápida sem planejamento completo

Comando	Categoria	Descrição
/gsd:progress	Status	Ver progresso geral do projeto
/gsd:health	Status	Verificar saúde do projeto
/gsd:add-todo	Tasks	Adicionar TODO ao projeto
/gsd:check-todos	Tasks	Verificar TODOs pendentes
/gsd:add-tests	Quality	Adicionar testes para uma feature
/gsd:verify-work	Quality	Verificar trabalho realizado
/gsd:map-codebase	Análise	Mapear estrutura do codebase
/gsd:cleanup	Manutenção	Limpar e organizar código
/gsd:pause-work	Sessão	Pausar trabalho (salvar contexto)
/gsd:resume-work	Sessão	Retomar trabalho pausado

Agents Especializados (12 agents)

Agent	Função
gsd-planner	Cria planos de implementação detalhados
gsd-executor	Executa implementações seguindo o plano
gsd-verifier	Verifica que a implementação está correta
gsd-debugger	Debugging estruturado e análise de erros
gsd-phase-researcher	Pesquisa documentação para uma fase específica
gsd-project-researcher	Pesquisa geral sobre o projeto
gsd-research-synthesizer	Sintetiza resultados de pesquisa
gsd-codebase-mapper	Mapeia e analisa estrutura do código
gsd-roadmapper	Cria roadmaps de desenvolvimento
gsd-plan-checker	Valida qualidade dos planos (read-only)
gsd-integration-checker	Verifica integrações entre componentes (read-only)
gsd-nyquist-auditor	Audita cobertura e qualidade do trabalho

6. superpowers — Biblioteca de Padrões de Desenvolvimento

Repositório: obra/superpowers | Versão: 5.0.2

Local: ~/.claude/plugins/marketplaces/obra/

O que faz

Biblioteca core de skills que ensinam ao Claude padrões comprovados de desenvolvimento: TDD, debugging sistemático, code review, uso de git worktrees, brainstorming, e mais. As skills são injetadas automaticamente no início de cada sessão via hook de SessionStart.

Skills (14 skills)

Skill	Descrição	Quando Usar
test-driven-development	Escrever testes primeiro, depois implementar. Red → Green → Refactor	Sempre que for implementar nova funcionalidade
systematic-debugging	Metodologia estruturada para encontrar e corrigir bugs	Quando encontrar um bug — ao invés de tentativa e erro
subagent-driven-development	Delegar tarefas para subagentes especializados em paralelo	Tasks complexas que podem ser divididas
dispatching-parallel-agents	Lançar múltiplos agentes em paralelo para máxima eficiência	Pesquisa, testes, refatoração em múltiplos arquivos
writing-plans	Criar planos de implementação detalhados antes de codificar	Features complexas que precisam de planejamento
executing-plans	Executar planos fase por fase com verificação	Após criar um plano com write-plan
brainstorming	Técnicas estruturadas de brainstorming para ideação	Explorar soluções alternativas para um problema
requesting-code-review	Solicitar review de código de outro agente	Antes de fazer merge de código importante
receiving-code-review	Processar e aplicar feedback de code review	Ao receber review — aplicar melhorias sugeridas
verification-before-completion	Verificar que tudo funciona antes de finalizar	Sempre, antes de marcar uma task como completa
finishing-a-development-branch	Processo completo para finalizar uma branch	Ao concluir trabalho em uma feature branch

Skill	Descrição	Quando Usar
using-git-worktrees	Usar git worktrees para trabalhar em múltiplas branches	Quando precisa alternar entre branches frequentemente
writing-skills	Criar novas skills para o Claude Code	Quando quiser ensinar algo novo ao Claude
using-superpowers	Meta-skill: como usar o sistema de superpowers	Referência sobre como invocar outras skills

Comandos

- **/brainstorm** — Inicia sessão de brainstorming estruturado
- **/write-plan** — Cria plano de implementação detalhado
- **/execute-plan** — Executa plano existente fase por fase

7. awesome-claude-code — Comandos da Comunidade

Repositório: [hesreallyhim/awesome-claude-code](https://github.com/hesreallyhim/awesome-claude-code)

Local: `~/claude/commands/awesome/`

O que faz

Coleção curada de 23 slash commands criados pela comunidade para tarefas comuns de desenvolvimento. Cada comando é um prompt otimizado para uma tarefa específica.

Comandos (/awesome:*)

Comando	Categoria	Descrição
<code>/awesome:commit</code>	Git	Criar commits bem formatados com mensagens descritivas
<code>/awesome:create-pr</code>	Git	Criar Pull Requests com título, descrição e test plan
<code>/awesome:create-pull-request</code>	Git	Variação alternativa de criação de PR
<code>/awesome:pr-review</code>	Git	Revisar um Pull Request com feedback estruturado
<code>/awesome:fix-github-issue</code>	Git	Resolver uma issue do GitHub com implementação
<code>/awesome:update-branch-name</code>	Git	Renomear branch seguindo convenções
<code>/awesome:act</code>	Código	Agir como um especialista específico
<code>/awesome:clean</code>	Código	Limpar e organizar código existente
<code>/awesome:optimize</code>	Código	Otimizar código para performance
<code>/awesome:todo</code>	Código	Gerenciar TODOs no código
<code>/awesome:create-prd</code>	Projeto	Criar Product Requirements Document
<code>/awesome:create-prp</code>	Projeto	Criar Product Requirements Plan
<code>/awesome:create-jtbd</code>	Projeto	Criar Jobs-To-Be-Done framework
<code>/awesome:create-hook</code>	Projeto	Criar hook do Claude Code
<code>/awesome:release</code>	Projeto	Preparar release com changelog

Comando	Categoria	Descrição
/awesome:context-prime	Contexto	Preparar contexto antes de uma tarefa
/awesome:initref	Contexto	Inicializar referências do projeto
/awesome:load-llms-txt	Contexto	Carregar llms.txt para contexto
/awesome:add-to-changelog	Docs	Adicionar entrada ao changelog
/awesome:update-docs	Docs	Atualizar documentação do projeto
/awesome:create-worktrees	Infra	Configurar git worktrees
/awesome:husky	Infra	Configurar husky para git hooks
/awesome:testing_plan_integration	Quality	Criar plano de testes de integração

8. ui-ux-pro-max — Inteligência de Design UI/UX

Repositório: [nextlevelbuilder/ui-ux-pro-max-skill](#) | Versão: 2.2.1

O que faz

Sistema completo de inteligência de design UI/UX com databases pesquisáveis de estilos, paletas de cores, tipografia, charts e guidelines para 13+ stacks tecnológicos. Ativado automaticamente quando o Claude detecta contexto de UI/design.

Skills (7 skills)

Skill	Recursos	Descrição	Quando Usar
ui-ux-pro-max	67 estilos, 96 paletas, 57 fontes, 25 charts	Core de design — database pesquisável de estilos, cores, tipografia, charts, UX guidelines	Ao criar qualquer interface: website, dashboard, app, landing page
ui-styling	Tailwind, shadcn/ui, 30+ fontes canvas	Estilização com Tailwind CSS, shadcn/ui components, responsive design, canvas design system	Estilizar componentes com Tailwind/shadcn
design	Logo, icon, CIP design	Design de logos, ícones e Corporate Identity Packages com scripts de geração e busca	Criar identidade visual, logos, ícones
design-system	Tokens, components, slides	Design tokens, component specs, geração de slides com layout patterns e dados CSV	Criar/manter design system, gerar apresentações
brand	Guidelines, voice, messaging	Brand guidelines completas: identidade visual, paleta de cores, tipografia, voz da marca	Criar ou aplicar brand guidelines
slides	Copywriting, layouts, estratégias	Criação de apresentações com fórmulas de copywriting, layout patterns e template HTML	Criar apresentações profissionais
banner-design	Tamanhos e estilos	Referência de tamanhos padrão de banners (social media, ads, web) e estilos	Criar banners para diferentes plataformas

Estilos Suportados

Glassmorphism · Claymorphism · Minimalism · Brutalism · Neumorphism · Bento Grid · Dark Mode · Skeuomorphism · Flat Design · Material Design · e 57+ mais

Stacks Suportados

React · Next.js · Astro · Vue · Nuxt.js · Nuxt UI · Svelte · SwiftUI · React Native · Flutter · Tailwind CSS · shadcn/ui · Jetpack Compose

9. notebooklm-py — Google NotebookLM

Repositório: [teng-lin/notebooklm-py](#) | Versão: 0.3.4

CLI: notebooklm | Skill: [.claude/skills/notebooklm/](#)

O que faz

Acesso programático completo ao Google NotebookLM — incluindo funcionalidades não disponíveis na interface web. Permite criar notebooks, adicionar fontes, gerar podcasts, vídeos, reports, quizzes e mais, tudo via CLI.

Comandos Principais

Comando	Descrição
notebooklm login	Autenticar com Google OAuth (abre browser)
notebooklm create "Título"	Criar novo notebook
notebooklm source add "URL"	Adicionar fonte (URL, YouTube, PDF, áudio, vídeo, imagem)
notebooklm source add-research "query"	Web research automático
notebooklm ask "pergunta"	Chat com as fontes do notebook
notebooklm generate audio "instrução"	Gerar podcast (deep-dive, brief, critique, debate)
notebooklm generate video "instrução"	Gerar vídeo (explainer, brief, múltiplos estilos)
notebooklm generate report --format study-guide	Gerar relatório (briefing-doc, study-guide, blog-post)
notebooklm generate quiz	Gerar quiz (easy/medium/hard)
notebooklm generate flashcards	Gerar flashcards para estudo
notebooklm generate slide-deck	Gerar apresentação de slides
notebooklm generate mind-map	Gerar mapa mental (instantâneo)
notebooklm download audio ./podcast.mp3	Baixar podcast como MP3
notebooklm download video ./video.mp4	Baixar vídeo como MP4
notebooklm download slide-deck ./slides.pptx --format pptx	Baixar slides como PPTX

Comando	Descrição
<code>notebooklm download quiz --format markdown quiz.md</code>	Baixar quiz como Markdown
<code>notebooklm language set pt_BR</code>	Definir idioma dos artefatos

Pré-requisito: Execute 'notebooklm login' antes de qualquer outro comando. Re-execute se comandos falharem com erro de autenticação.

10. Anthropic Skills — Coleção Oficial

Repositório: anthropics/skills | Local: .claude/skills/

O que faz

Coleção oficial de skills criadas pela Anthropic. São as skills mais bem testadas e otimizadas, cobrindo geração de documentos, design, desenvolvimento e produtividade.

Skills (17 skills)

Skill	Categoria	Descrição	Quando Usar
pdf	Documentos	Gerar PDFs profissionais com layouts, tabelas, gráficos e formulários	Relatórios, guias, documentação formal
docx	Documentos	Criar e editar documentos Word (.docx) com formatação completa	Documentos para compartilhar com equipe
xlsx	Documentos	Criar e editar planilhas Excel (.xlsx) com fórmulas e gráficos	Planilhas de dados, análises, relatórios
pptx	Documentos	Criar e editar apresentações PowerPoint (.pptx)	Apresentações, pitch decks, treinamentos
claude-api	Desenvolvimento	Construir apps com Claude API em 7 linguagens (Python, TS, Go, Java, PHP, Ruby, C#)	Integrar Claude em aplicações
mcp-builder	Desenvolvimento	Construir servidores MCP com docs de referência e scripts	Criar novos MCP servers
skill-creator	Desenvolvimento	Criar novas skills com agents, eval viewer e templates	Desenvolver suas próprias skills
web-artifacts-builder	Desenvolvimento	Construir artefatos web interativos (HTML/CSS/JS self-contained)	Criar demos, protótipos, tools web
webapp-testing	Quality	Testar web applications com exemplos e scripts de automação	Testes end-to-end, smoke tests
frontend-design	Design	Padrões de design frontend — layouts, componentes, responsividade	Criar interfaces de qualidade
canvas-design	Design	Design baseado em canvas HTML5 com 30+ fontes bundled	Criar gráficos, infográficos, visual content
brand-guidelines	Design	Criar e aplicar brand guidelines consistentes	Padronizar identidade visual

Skill	Categoria	Descrição	Quando Usar
theme-factory	Design	Gerar temas customizados com showcase PDF e templates	Criar temas para apps/sites
algorithmic-art	Criativo	Criar arte algorítmica/generativa com templates JS	Arte digital, backgrounds, visual effects
doc-coauthoring	Produtividade	Co-autoria de documentos — edição colaborativa estruturada	Escrever documentos longos iterativamente
internal-comms	Produtividade	Comunicações internas — emails, anúncios, newsletters	Escrever comunicações profissionais
slack-gif-creator	Produtividade	Criar GIFs animados para Slack com Python	Criar GIFs customizados para equipe

11. Context7 — Documentação em Tempo Real

Repositório: upstash/context7 | CLI: ctx7 v0.3.5

O que faz

Busca documentação atualizada de qualquer biblioteca/framework em tempo real. Ao invés de depender do conhecimento de treinamento (que pode estar desatualizado), o Context7 consulta a documentação mais recente diretamente.

Quando usar

- Sempre que perguntar sobre APIs, configurações ou código de bibliotecas externas
- Quando a versão da biblioteca importa (ex: "React 19", "Next.js 15")
- Para verificar breaking changes ou APIs deprecated
- Quando o Claude não tem certeza sobre uma API ou configuração

Componentes

Componente	Como Usar	Descrição
CLI ctx7	ctx7 library react "hooks" → ctx7 docs /facebook/react "useEffect"	Busca via linha de comando — 2 passos: resolver ID + buscar docs
MCP Server	Automático via tools resolve-library-id e query-docs	Integração direta via MCP — Claude usa automaticamente
Skill find-docs	Ativação automática ao perguntar sobre libs	Instrui o Claude a usar ctx7 CLI para buscar docs
Skill context7-mcp	Ativação automática	Instrui o Claude a usar MCP tools para buscar docs
/docs	Manual: /docs react hooks	Comando rápido para buscar documentação
Agent docs-researcher	Delegação automática	Subagente leve que busca docs sem poluir contexto principal

Exemplo de Uso

```
# Passo 1: Resolver biblioteca
ctx7 library nextjs "app router middleware"
# → /vercel/next.js (Score: 95, Snippets: 2847)

# Passo 2: Buscar documentação
ctx7 docs /vercel/next.js "How to add auth middleware to app router"
# → Retorna code snippets + info snippets atualizados
```

Funciona sem autenticação. Para limites maiores: `ctx7 login` ou defina `CONTEXT7_API_KEY`.

12. Ralph — PRDs e Execução Autônoma

Repositório: [snarktank/ralph](#) | Versão: 1.0.0

O que faz

Sistema para gerar PRDs (Product Requirements Documents) e convertê-los em formato JSON executável pelo loop autônomo do Ralph. O Ralph trabalha em um ciclo: PRD → prd.json → execução autônoma → verificação.

Skills

Skill	Trigger	Fluxo	Quando Usar
/prd	"crie um PRD", "planeje feature"	1. Recebe descrição da feature 2. Faz 3-5 perguntas de clarificação 3. Gera PRD estruturado 4. Salva em tasks/prd-[nome].md	Ao iniciar uma nova feature — antes de codificar
/ralph	"converta este PRD", "ralph json"	1. Recebe PRD (markdown ou texto) 2. Extrai user stories, tasks, critérios de aceitação 3. Gera prd.json com formato executável 4. Salva no diretório ralph	Quando tem um PRD pronto e quer execução autônoma

13. ruflo/claude-flow — Orquestração Enterprise

Repositório: [ruvnet/ruflo](https://github.com/ruflo/claude-flow) | Versão: 2.5.0

O que faz

Plataforma enterprise de orquestração de agentes IA com 150+ comandos, 74+ agents, SPARC methodology, coordenação de swarms, GitHub automation e neural training. É o sistema mais completo e complexo instalado.

Categorias de Skills (38 skills)

Categoria	Skills	Descrição
Swarm	swarm-orchestration, swarm-advanced	Coordenação de múltiplos agentes (Hierarchical, Mesh, Ring, Star)
GitHub	github-code-review, github-multi-repo, github-project-management, github-release-management, github-workflow-automation	Automação completa de GitHub — PRs, reviews, releases, projetos
SPARC	sparc-methodology	Specification, Pseudocode, Architecture, Refinement, Completion — 18 modos
Pair Programming	pair-programming	Programação em par com agente IA
AgentDB	agentdb-advanced, agentdb-learning, agentdb-memory-patterns, agentdb-optimization, agentdb-vector-search	Database de agentes com aprendizado e busca vetorial
Hive Mind	hive-mind-advanced	Inteligência coletiva de agentes
Flow Nexus	flow-nexus-neural, flow-nexus-platform, flow-nexus-swarm	Plataforma cloud de orquestração (70+ tools)
Performance	performance-analysis, worker-benchmarks	Análise de performance e benchmarking
Hooks	hooks-automation	Automação de hooks do Claude Code
Reasoning	reasoningbank-agentdb, reasoningbank-intelligence	Banco de raciocínio e inteligência artificial

Categoria	Skills	Descrição
Quality	verification-quality, worker-integration	Verificação de qualidade e integração
v3	v3-cli-modernization, v3-core-implementation, v3-ddd-architecture, + 6 mais	Modernização, DDD, MCP optimization, security, memory

MCP Server — claude-flow

O MCP server claude-flow fornece 40+ tools para coordenação de swarm. Configurado em ~/.mcp.json com comando: `npx -y claude-flow@alpha mcp start`

14. Base de Dados — Referência Rápida

Todas as 35 Skills do Projeto

#	Skill	Origem	Tipo
1	algorithmic-art	Anthropic	Arte generativa
2	banner-design	UI/UX Pro Max	Design de banners
3	brand	UI/UX Pro Max	Brand guidelines
4	brand-guidelines	Anthropic	Brand guidelines
5	canvas-design	Anthropic	Design canvas
6	claude-api	Anthropic	Desenvolvimento API
7	context7-cli	Context7	Docs lookup CLI
8	context7-mcp	Context7	Docs lookup MCP
9	defuddle	Obsidian	Web scraping limpo
10	design	UI/UX Pro Max	Logo/icon design
11	design-system	UI/UX Pro Max	Design tokens
12	doc-coauthoring	Anthropic	Co-autoria
13	docx	Anthropic	Documentos Word
14	find-docs	Context7	Docs lookup
15	frontend-design	Anthropic	Frontend patterns
16	internal-comms	Anthropic	Comunicação interna
17	json-canvas	Obsidian	JSON Canvas files
18	mcp-builder	Anthropic	Criar MCP servers
19	notebooklm	NotebookLM	Google NotebookLM
20	obsidian-bases	Obsidian	Obsidian Bases
21	obsidian-cli	Obsidian	Obsidian CLI
22	obsidian-markdown	Obsidian	Obsidian Markdown

#	Skill	Origem	Tipo
23	pdf	Anthropic	Geração de PDFs
24	pptx	Anthropic	Apresentações PPT
25	prd	Ralph	Product Requirements
26	ralph	Ralph	PRD → JSON executável
27	skill-creator	Anthropic	Criar skills
28	slack-gif-creator	Anthropic	GIFs para Slack
29	slides	UI/UX Pro Max	Criação de slides
30	theme-factory	Anthropic	Gerar temas
31	ui-styling	UI/UX Pro Max	Tailwind/shadcn
32	ui-ux-pro-max	UI/UX Pro Max	Core design DB
33	web-artifacts-builder	Anthropic	Artefatos web
34	webapp-testing	Anthropic	Testes web apps
35	xlsx	Anthropic	Planilhas Excel

Todos os MCP Servers

Server	Comando	Ferramentas	Propósito
n8n-mcp	<code>npx -y n8n-mcp</code>	search, validate, create workflows	Automação de workflows n8n
context7	<code>npx -y @upstash/context7-mcp</code>	resolve-library-id, query-docs	Documentação em tempo real
claude-flow	<code>npx -y claude-flow@alpha mcp start</code>	40+ tools de swarm	Orquestração de agentes

Todos os CLIs Instalados

CLI	Versão	Comando de Verificação	Propósito
ctx7	0.3.5	<code>ctx7 --version</code>	Busca de documentação Context7
notebooklm	0.3.4	<code>notebooklm --version</code>	Google NotebookLM CLI
bun	1.3.9	<code>bun --version</code>	Runtime JavaScript (usado pelo claude-mem)

15. Compatibilidade por Plataforma

Plugin/Skill	Claude Code	Claude AI (web)	Claude Cowork
claude-mem	■ Completo	■ Não compatível	■■ Se tiver ~/.claude/
n8n-mcp	■ Completo	■ Não compatível	■■ Se tiver MCP
obsidian-skills	■ Completo	■ Não compatível	■ Via .claude/skills/
get-shit-done	■ Completo	■ Não compatível	■■ Se tiver ~/.claude/
superpowers	■ Completo	■ Não compatível	■■ Se tiver plugins/
awesome-claude-cod e	■ Completo	■ Não compatível	■■ Se tiver commands/
ui-ux-pro-max	■ Completo	■ Não compatível	■ Via .claude/skills/
notebooklm-py	■ Completo	■ Não compatível	■■ Se tiver pip install
Anthropic Skills	■ Completo	■■ Algumas built-in	■ Via .claude/skills/
Context7	■ Completo	■ Não compatível	■■ Se tiver MCP + CLI
Ralph	■ Completo	■ Não compatível	■ Via .claude/skills/
ruflo/claude-flow	■ Completo	■ Não compatível	■■ Se tiver plugins/

Como instalar no Claude Cowork

O Claude Cowork roda sobre o Claude Code. Para que os plugins funcionem no Cowork:

1. **Skills de projeto** (.claude/skills/) — já estão no repositório, funcionam automaticamente
2. **Plugins globais** (~/.claude/plugins/) — precisam estar no filesystem do ambiente
3. **MCP Servers** (~/.mcp.json) — precisam ser configurados no ambiente
4. **CLIs** (ctx7, notebooklm) — precisam ser instalados: `npm i -g ctx7@latest`; `pip install notebooklm-py`
5. **Comandos globais** (~/.claude/commands/) — copiar os .md para o diretório correto

O Claude AI (web — claude.ai) NÃO suporta skills, plugins, hooks, MCP servers ou comandos customizados. Essas funcionalidades são exclusivas do Claude Code.

Documento gerado automaticamente em 14/03/2026 às 20:19.
Total: 12 repositórios · 35 skills · 55+ comandos · 90+ agents · 3 MCP servers